

Architecture_SIH26162_FireVigil

Technical Architecture Document

FireVigil — AI-Based Detection and Classification of Industrial Fires and Persistent Thermal Sources Using NASA FIRMS, OSM & Satellite Data

1. Architecture Overview

2. Technology Stack

2.1 Frontend

2.2 Backend (Core API)

2.3 Classification / Geospatial Microservice

2.4 Data Layer

2.5 DevOps & Infrastructure

3. Data Model (Core Entities)

4. API Design (REST, versioned /api/v1)

5. Processing Pipeline (Step-by-Step)

6. Security Considerations

7. Scalability & Reliability Notes

8. Deployment Diagram (Demo Setup)

9. Repository Structure (suggested monorepo)

10. Key Risks & Mitigations

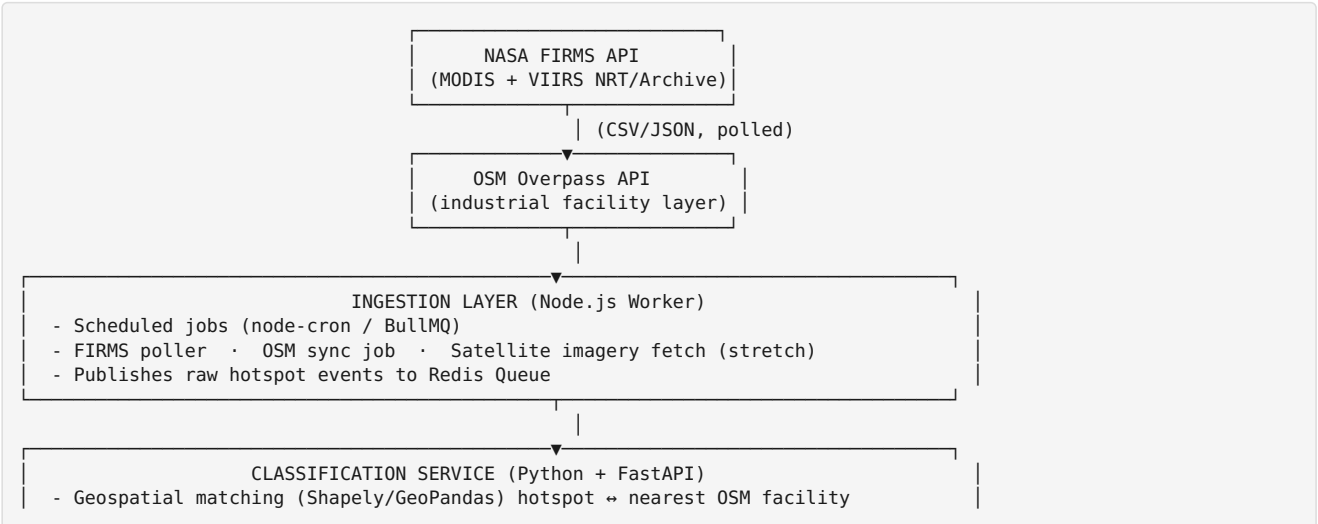
Technical Architecture Document

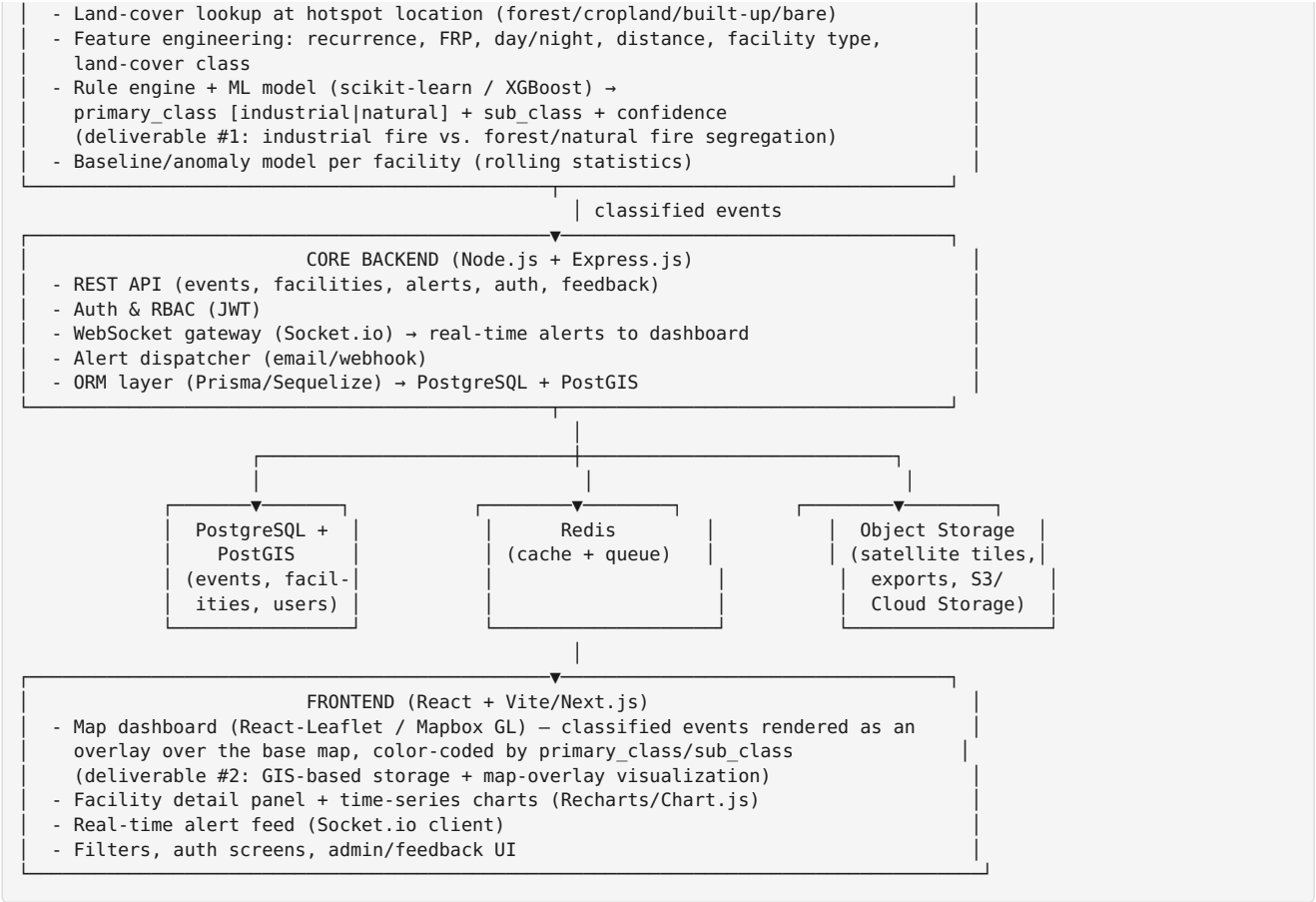
FireVigil — AI-Based Detection and Classification of Industrial Fires and Persistent Thermal Sources Using NASA FIRMS, OSM & Satellite Data

SIH26162 | Theme: Disaster Management | Sponsor: National Technical Research Organisation (NTRO) | Dataset: firms.modaps.eosdis.nasa.gov/map

1. Architecture Overview

The system is a **modular, service-oriented web application** with a scheduled ingestion pipeline, a classification engine, a REST API, and a React-based geospatial dashboard. Node.js/Express.js form the backbone (API gateway, auth, orchestration, real-time layer), while a dedicated **Python microservice** handles the geospatial/ML-heavy classification work — this keeps the stack polyglot-appropriate without abandoning the required Node/Express/React core.





2. Technology Stack

2.1 Frontend

Layer	Technology	Rationale
Framework	React 18 (with Vite for fast dev/build)	Required stack; fast HMR for hackathon iteration
Mapping	React-Leaflet (OpenStreetMap tiles) or Mapbox GL JS	Leaflet is free/open-source and pairs naturally with OSM data already in use
Charts	Recharts	Simple, React-native charting for time-series thermal activity
State management	React Query (TanStack Query) + Context/Zustand	Server-state caching for API calls, lightweight local state
Real-time	Socket.io-client	Live alert push without polling
Styling	Tailwind CSS	Rapid, consistent UI build under hackathon time pressure
Auth	JWT stored in httpOnly cookie / secure storage	Simplicity + security balance

2.2 Backend (Core API)

Layer	Technology	Rationale
Runtime	Node.js 20 LTS	Required stack
Framework	Express.js	Required stack; mature middleware ecosystem
Real-time	Socket.io	Push classified alerts to dashboard instantly

Auth	JWT (jsonwebtoken) + bcrypt for password hashing	Standard, stateless auth suitable for RBAC
Validation	Zod or Joi	Request schema validation
ORM	Prisma (or Sequelize)	Type-safe DB access, migrations
Job scheduling	node-cron for periodic polling; BullMQ (Redis-backed) for queued/retryable ingestion & classification jobs	FIRMS/OSM polling needs resilience & backoff
Logging	Winston / Pino	Structured logs for ingestion & alert auditability

2.3 Classification / Geospatial Microservice

Layer	Technology	Rationale
Framework	Python 3.11 + FastAPI	Best-in-class geospatial & ML libraries live in Python; FastAPI is fast and easy to containerize alongside Node services
Geospatial	GeoPandas, Shapely, PyProj	Spatial join of hotspots to OSM polygons, distance calculations
ML	scikit-learn (baseline: Random Forest / Gradient Boosting classifier)	Interpretable, fast to train on limited labeled hackathon data; can be swapped for XGBoost/LightGBM
Data handling	Pandas, NumPy	Feature engineering, rolling statistics per facility
Optional CV	PyTorch / OpenCV (stretch)	If satellite image classification (visual fire confirmation) is attempted
Inter-service comms	Internal REST call from Node backend, or shared Redis queue	Node orchestrates, Python does the heavy lifting

Why not do classification purely in Node? Geospatial analysis (spatial joins, polygon operations) and ML tooling are dramatically more mature in Python (GeoPandas/Shapely/scikit-learn have no equally robust Node equivalents). Isolating this as a microservice also lets it scale/redeploy independently and keeps the Node/Express layer focused on API orchestration, auth, and real-time delivery — which is where Node genuinely excels.

2.4 Data Layer

Component	Technology	Rationale
Primary DB	PostgreSQL 16 + PostGIS extension	Native geospatial indexing (GiST) for fast “nearest facility” and radius queries — essential for hotspot↔facility matching at scale
Cache / Queue	Redis	BullMQ job queue, API response caching, rate limiting
Object storage	AWS S3 / Cloudflare R2 / GCP Cloud Storage	Satellite imagery tiles, CSV export files
Search (optional)	Elasticsearch/Meilisearch (stretch)	Fast facility name/text search if dataset grows large

2.5 DevOps & Infrastructure

Component	Technology
Containerization	Docker + docker-compose (local/dev), Kubernetes optional for production scale
CI/CD	GitHub Actions (lint, test, build, deploy)

Hosting (demo)	Frontend on Vercel/Netlify ; Node API on Render/Railway/AWS ECS ; Python service co-located as a container; DB on Supabase/Neon (Postgres+PostGIS) or self-hosted
Monitoring	Prometheus + Grafana (stretch) or simple health-check endpoints for demo
Secrets	<code>.env</code> + platform secret manager (never committed)
API docs	Swagger/OpenAPI (via <code>swagger-jsdoc</code> + <code>swagger-ui-express</code>)

3. Data Model (Core Entities)

User
- id, name, email, password_hash, role [admin analyst viewer], created_at
Facility (from OSM)
- id, osm_id, name, type [refinery power_plant kiln mine landfill other], geometry (PostGIS geometry), state, district, source, last_synced_at
Hotspot (raw FIRMS record)
- id, latitude, longitude, acq_date, acq_time, satellite, instrument (MODIS VIIRS), confidence, brightness, frp (fire radiative power), daynight, raw_payload(jsonb)
ClassifiedEvent
- id, hotspot_id (FK), facility_id (FK, nullable), primary_class [industrial natural], -- core official deliverable #1 sub_class [industrial_fire gas_flare agricultural_burning mining_activity forest_fire other_natural], land_cover_type (from land-cover dataset, e.g. forest cropland built_up bare), confidence_score, model_version, distance_to_facility_m, is_anomalous (bool), created_at
FacilityBaseline
- id, facility_id (FK), avg_daily_detections, avg_frp, std_dev_frp, window_start, window_end, updated_at
Alert
- id, classified_event_id (FK), severity [high medium low], status [new acknowledged resolved false_positive], sent_at, acknowledged_by (FK User)
Feedback
- id, classified_event_id (FK), user_id (FK), corrected_label, notes, created_at

4. API Design (REST, versioned `/api/v1`)

Method	Endpoint	Description
POST	<code>/api/v1/auth/login</code>	Authenticate, return JWT
POST	<code>/api/v1/auth/register</code>	Create analyst/admin account (admin-only)
GET	<code>/api/v1/hotspots</code>	List raw hotspots (filter by date, bbox)
GET	<code>/api/v1/events</code>	List classified events (filter by class, region, date, facility)
GET	<code>/api/v1/events/:id</code>	Event detail with facility + history
GET	<code>/api/v1/facilities</code>	List facilities (filter by type, region)
GET	<code>/api/v1/facilities/:id</code>	Facility detail + time-series activity
GET	<code>/api/v1/facilities/:id/timeseries</code>	Historical thermal activity for charts
GET	<code>/api/v1/alerts</code>	List alerts (filter by severity/status)
PATCH	<code>/api/v1/alerts/:id</code>	Update alert status (acknowledge/resolve)
POST	<code>/api/v1/events/:id/feedback</code>	Analyst correction → feeds retraining set
GET	<code>/api/v1/export</code>	CSV/JSON export of filtered events

WS	/socket.io (namespace /alerts)	Real-time push of new classified high-priority events
----	---------------------------------	---

Internal (Node → Python microservice, not public): | Method | Endpoint | Description | |—|—| | POST |
 /internal/classify | Batch classify a set of raw hotspots + candidate facilities | | POST | /internal/baseline/recompute |
 Recompute rolling baselines for facilities |

5. Processing Pipeline (Step-by-Step)

1. **Scheduled ingestion job** (node-cron, every ~15–60 min matching FIRMS NRT cadence) fetches new hotspots for the configured bounding box/country via FIRMS API (CSV/JSON), for both MODIS and VIIRS sources.
2. Raw hotspots are **deduplicated** and inserted into `hotspot` table; new records are pushed to a BullMQ queue (`classification-queue`).
3. **OSM sync job** (daily/weekly, less frequent) refreshes the `facility` table via Overpass API queries scoped to the area of interest (industrial land use tags).
4. A **worker** consumes the classification queue, batches hotspots, and calls the **Python FastAPI classification service**.
5. The classification service:
 - Performs a **spatial nearest-neighbor join** (PostGIS `ST_DWithin` / `ST_Distance` or GeoPandas `sjoin_nearest`) between each hotspot and known facilities.
 - Computes features: distance to nearest facility, historical recurrence count at that location, FRP/brightness, day/night flag, facility type.
 - Applies **rule-based pre-filtering** (e.g., “detected at same coords >20 times in 90 days” → likely persistent source) combined with the **ML classifier** for nuanced cases.
 - Returns classification + confidence back to Node.
6. Node backend **persists** the `ClassifiedEvent`, updates `FacilityBaseline` stats, and — if classified as `industrial_fire` with high confidence and low recurrence (i.e., anomalous) — **triggers an Alert**.
7. Alert dispatcher pushes via **Socket.io** to connected dashboards and optionally sends **email/webhook**.
8. **React dashboard** subscribes to the socket channel and REST API, rendering the map, facility panel, and alert feed in real time.
9. Analysts can **submit feedback/corrections**, stored for future model retraining (out of MVP scope to fully automate retraining, but the data pipeline supports it).

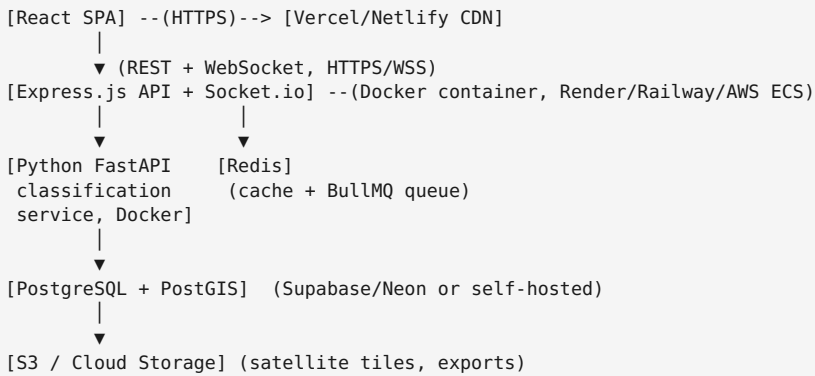
6. Security Considerations

- JWT access + refresh token flow; short-lived access tokens.
- Role-based access control: `admin` (manage users/config), `analyst` (view + give feedback + acknowledge alerts), `viewer` (read-only).
- Rate limiting on public API routes (`express-rate-limit`).
- Input validation on all endpoints (Zod schemas) to prevent injection.
- HTTPS enforced; CORS restricted to known frontend origin(s).
- Secrets (FIRMS API key, DB credentials, JWT secret) via environment variables, never committed to source control.
- Audit log table for classification/alert lifecycle events (who acknowledged what, when).

7. Scalability & Reliability Notes

- **PostGIS spatial indexes (GiST)** on `facility.geometry` and hotspot lat/long keep nearest-facility queries fast even as facility count grows to national scale.
- **BullMQ + Redis** decouples ingestion from classification, so a slow/failed classification run doesn’t block new data ingestion, and failed jobs can be retried with backoff.
- Classification microservice is **stateless** and can be horizontally scaled behind a load balancer if hotspot volume increases (e.g., pan-India rollout).
- FIRMS/OSM API outages are handled gracefully: ingestion jobs log failures, retry on next scheduled run, and the dashboard serves last-known-good cached data (`Redis` / `DB`) rather than failing.
- Historical archive growth is manageable by **partitioning the hotspot table by month** (Postgres native partitioning) once data volume grows.

8. Deployment Diagram (Demo Setup)



9. Repository Structure (suggested monorepo)

```
firevigil/
├── apps/
│   ├── web/           # React frontend (Vite)
│   ├── api/           # Express.js backend
│   └── classifier/     # Python FastAPI classification microservice
├── packages/
│   └── shared-types/   # Shared TS types/interfaces (API contracts)
├── infra/
│   ├── docker-compose.yml
│   └── k8s/ (optional)
├── docs/
│   ├── PRD.md
│   └── ARCHITECTURE.md
└── .github/workflows/  # CI/CD pipelines
```

10. Key Risks & Mitigations

Risk	Mitigation
FIRMS data latency/resolution too coarse for precise facility attribution	Combine with OSM buffer radius tuning + historical recurrence smoothing rather than relying on single-detection precision
Limited labeled training data for ML classifier within hackathon timeframe	Start with a strong rule-based classifier (recurrence + distance + FRP thresholds) as MVP baseline, layer ML on top as time allows, and seed synthetic/historical FIRMS archive data for labels
OSM facility data incompleteness in demo region	Pick a well-mapped pilot region for the demo; document limitation and roadmap for supplementary datasets (CPCB facility registry, state industrial directories)
Real-time expectations vs. FIRMS' actual update cadence (hours, not seconds)	Clearly communicate “near real-time” (aligned to satellite pass + FIRMS processing latency) in the UI/pitch, not literal live video-style monitoring